



NUST CHIP DESIGN CENTRE

Digital Design Verification

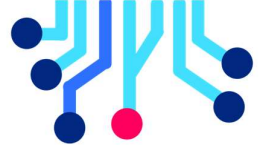
Weekly Task

Cache Design in SystemVerilog

Release: 1.0

Date: 14-June-2024

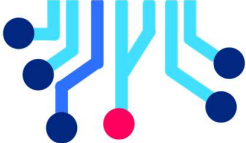
NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	14/06/2024	Ali Aqdas	Complete manual	-



Contents

Objective 3

Instructions 3

Downloading Files..... 4

Memory Initialization and Logs..... 4

Simulation 4

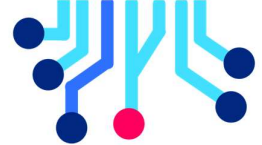
Cache Evaluation..... 4

General Requirements 4

Task 1: Direct Mapped Cache 4

Task 2: Set Associative Cache 4

Task 3: Fully Associative Cache 5



Objective

The objective of this lab is to:

- Understand and design different types of caches

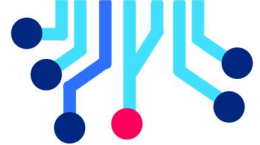
Tools

- Cadence Xcelium
- Python
- Linux

Instructions

The students are required to submit all design files and test benches.

./<student-name>/	./rtl/	direct_mapped.sv set_associative.sv fully_associative.sv
	./tb/	direct_mapped_tb.sv set_associative_tb.sv fully_associative_tb.sv



Downloading Files

A skeleton and a verification framework have been developed and uploaded to GitHub. The students are required to clone the repository from

```
$ git clone https://github.com/aaqdas/cache-tester.git
```

Memory Initialization and Logs

Now switch to cloned directory. The directory contains several script files in `scripts` which generate memory initialization files for caches in `init_files` and log files in `log`

`cache_init.py` is the primary script, that takes different flags and generates initialization files for different configurations of caches. It takes blocks `(-b)`, words per block `(-w)` and associativity `(-a)` as inputs.

```
$ python3 ./cache_init.py -b 256 -w 1 -a 1
```

Simulation

Designed caches can be simulated by integrating your design in available testbench in `tb` directory. The simulations must be run in Xcelium to prevent issues in porting.

The simulation testbench generates a log under the name `core_access.log` in the `log` directory.

Cache Evaluation

The cache results can be evaluated using comparison tool in `scripts` directory. The usage of the tool is as follows

```
$ python3 ./check.py ../log/access.log ../log/core_access.log
```

General Requirements

The designed caches must be byte addressable, i.e. each byte in the cache has an address associated with it. The inputs and outputs of the cache must follow the structure in the test bench and must output 32 - bit data on its data port (`data_o`) and generate a hit signal (`hit_o`) on every hit.

Task 1: Direct Mapped Cache

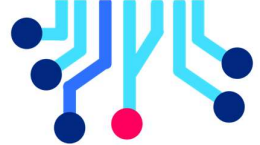
Design a direct mapped cache from the below specifications

```
Blocks = 256
Physical Address = 16 - bit
Words Per Block = 4
Word Size = 4 Bytes
```

Task 2: Set Associative Cache

Design a two-way set-associative cache from the below specifications

```
Blocks = 128
Associativity = 2
Physical Address = 16 - bit
Words Per Block = 4
```



Word Size = 4 Bytes

Task 3: Fully Associative Cache

Design a fully associative cache from the below specifications

Blocks = 1
Associativity = 256
Physical Address = 16 - bit
Words Per Block = 4
Word Size = 4 Bytes